

Architecture, Requirements & Design

Garth Watney
NASA Jet Propulsion Laboratory
February 9, 2026

Copyright © 2026 California Institute of Technology.
Government sponsorship acknowledged.

Jet Propulsion Laboratory
California Institute of Technology

Software Architecture

- Software Architecture is different from Software Design
- Software Architecture describes the skeleton and high level infrastructure of the software
  - Independent of the application domain
- Software Design describes the implementation of the domain within the software architecture
  - Breaks the software down into elements
  - Describes the purpose of each element
  - Describes the inner workings of each element
- Software Architecture is important
  - Antidote to software chaos
  - Glue and foundation that holds the software together
- Be vigilant against architectural erosion
  - Maintain the architectural integrity throughout development

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Software Architecture

- Examples of different software architecture
  - Pipes and Filters
  - Publish and Subscribe
  - Client-Server
  - Blackboard
  - Data-base
  - Event-driven
  - Component
- Classic JPL Flight Software Architecture
  - Multi-threaded module-based architecture
  - Modules only communicate through events using message queues
  - Static point to point connection
  - Monolithic
- Component based architecture
  - A component is a unit of computation with a well-defined interface
  - A component has no symbolic dependencies on other components
    - Compile, Load and Execute independently of other components
  - Components only communicate with each other through ports
  - Components encapsulate threads and queues and states

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Software Architectural Attributes

- Modularity
  - Modularity improves software development quality and maintainability
  - Decompose the software into a collection of modules (components or libraries)
  - A module is
    - A unit of work assigned to a developer
    - Has a well-defined set of requirements
    - Has a well-defined interface
    - Unit tested before being delivered into the integration build

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

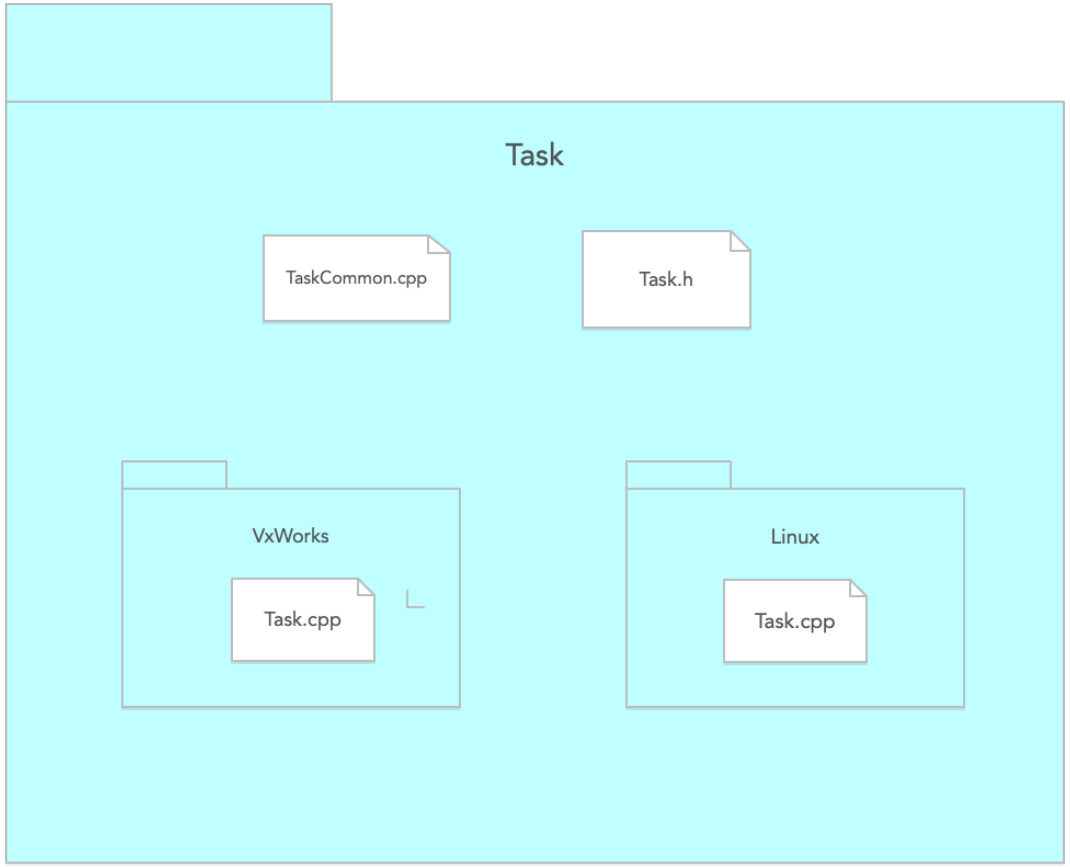
Software Architectural Attributes

- Module Coupling
  - The extent that modules (components) are related to each other
  - Examples of high coupling (bad)
    - Control coupling – one component controls the flow of another component by passing a “what-to-do” flag
    - Data coupling – components share a common data space
    - Content coupling – components share common code or data structures
- Module Cohesion
  - The extent that data and functions inside a module (component) belong to each other
  - Examples of high cohesion (good)
    - All the functions and data for a device driver pertain to the operation of the device
    - A Telemetry Manager component only processes telemetry channels and not commands
- Strive for Low Coupling and High Cohesion

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Software Architectural Attributes

- Portability
  - Software that is portable to a desktop workstation is significantly easier to develop.
  - Ensure your software is readily portable to your desktop workstation (Linux/Windows) and not just the embedded target
  - Hide Operating System differences in an OSAL (Operating System Adaptation Layer).
  - Avoid the use of scattered conditional compilations by creating different implementations of a class or function at the lowest level.



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.



Software Architectural Attributes

- Reusability
  - Use frameworks, libraries, algorithms, design patterns that are well tested and understood.
  - Fprime framework with its core components are an example of good reusability
  - Quantum Framework is a relatively simple and powerful framework for implementing hierarchical state-machines
  - “Design Patterns” by the “Gang of Four” present well understood software design patterns.

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Other Software Architectural Principles

- No dynamic memory allocation after initialization
  - Deterministic behavior
- No multiple class inheritance
- Limit class hierarchy
- Integrity checks
  - Asserts
- Performance
  - The software should perform well in a resource constrained environment.
- Keep it simple
  - If your code is complicated and ugly, it’s probably wrong.

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

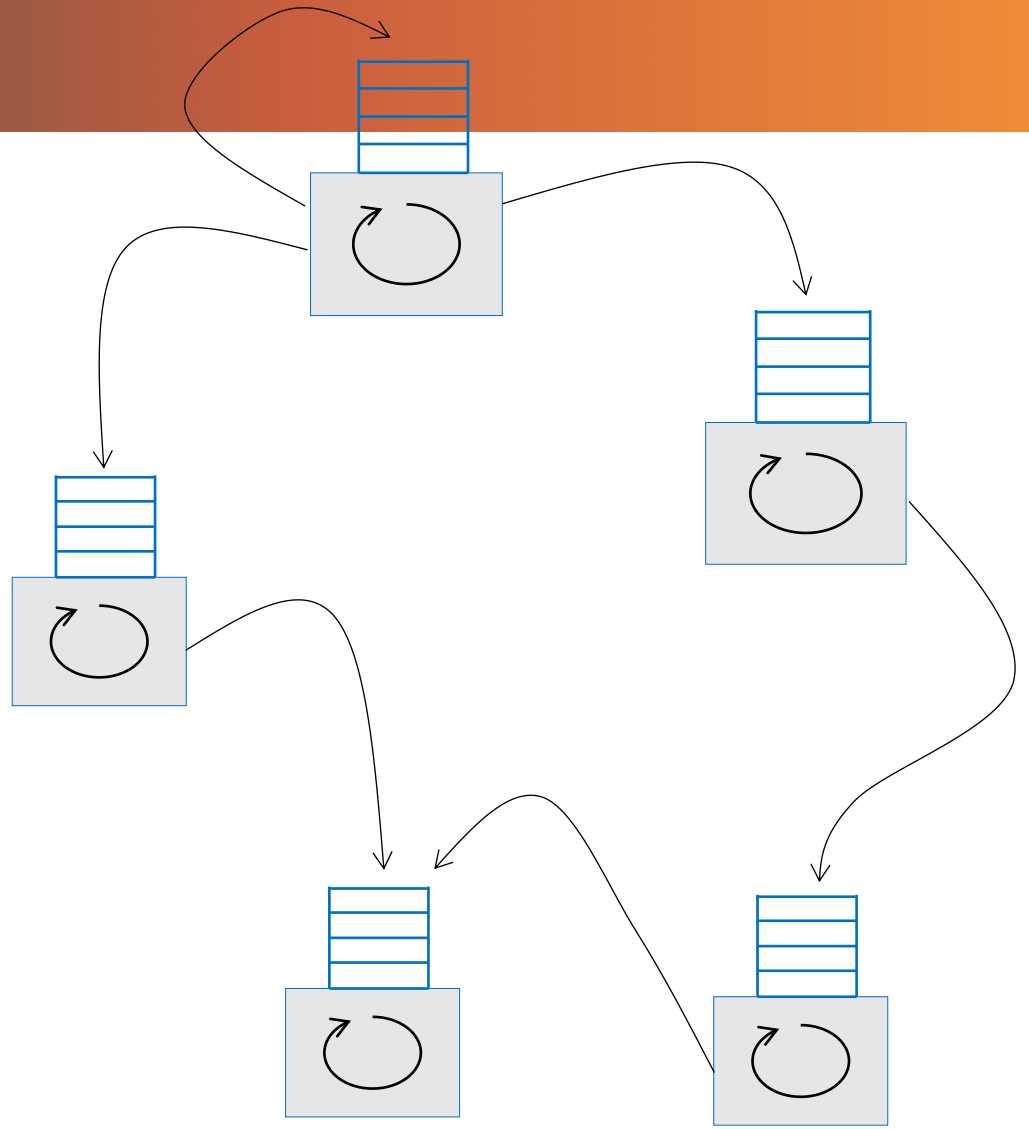
Software Architectural Views

- Software architecture is captured by different views or perspectives.
- These perspectives encompass the software architectural model
- These perspectives are not mutually exclusive

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

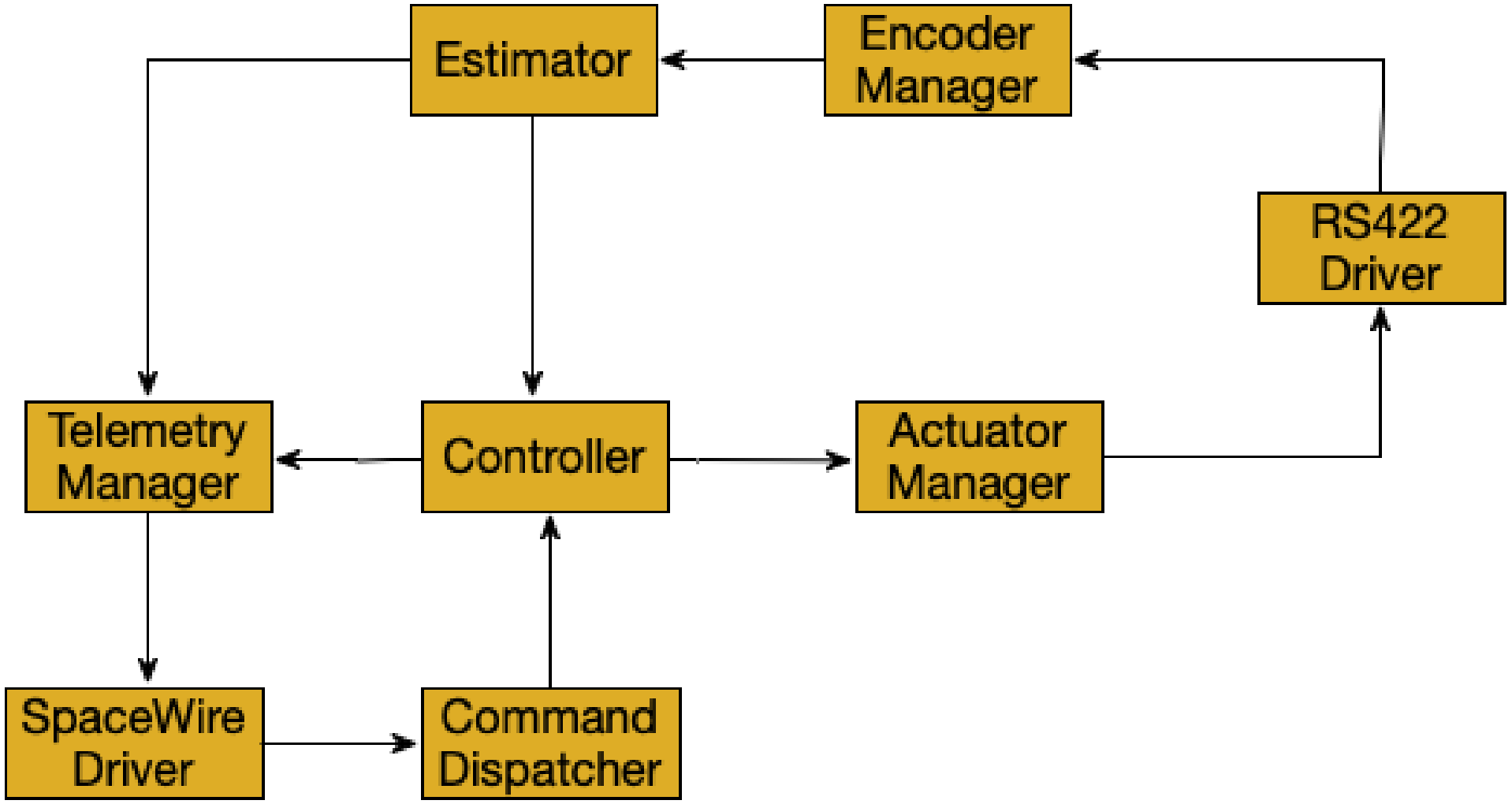
Architecture Views:
Software Task View

- Tasks are execution threads
- Tasks communicate via event messages which are placed on the task input queue
- Tasks sleep until a message arrives and then process events off their input queue
- Tasks have execution priority
- Tasks can be:
  - Rate-group driven (1 Hz, 10 Hz etc)
  - Data driven
  - Background (continuous low priority task)



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

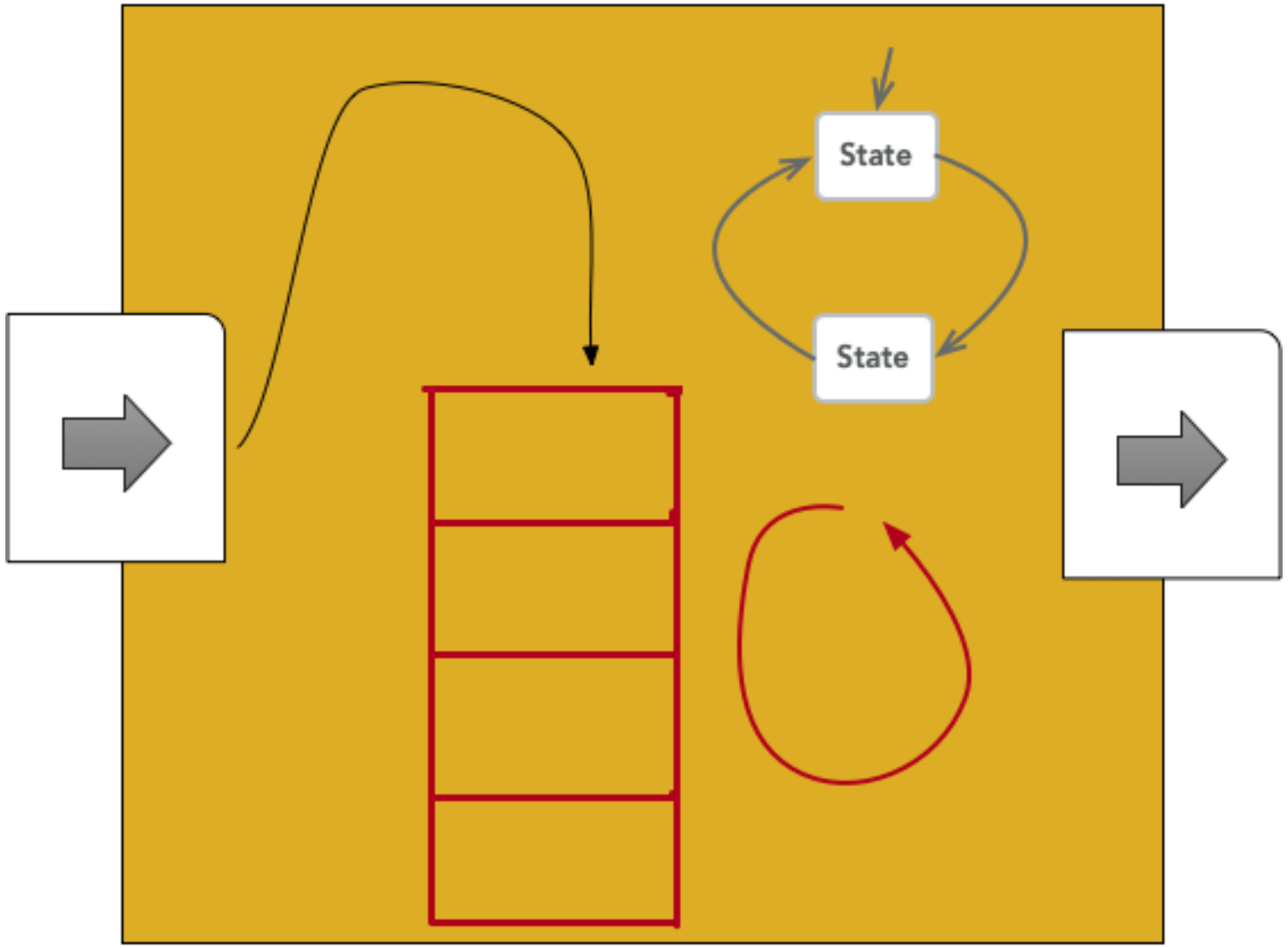
Architecture Views:
Component View



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Software Component Encapsulation

- A component encapsulate
  - A task
  - A state-machine
  - An input queue
  - Input and Output Ports



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Software Requirements

- Requirements are typically layered
  - Mission Requirements
  - Project Requirements
  - System Requirements
  - Subsystem Requirements
  - Flight software Requirements
  - Component Requirements
- Requirements are traced up and down
  - Higher level requirements are satisfied by lower level requirements
  - Lower level requirements are traced back to upper level requirements

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Software Requirements

- Focus on Flight Software Requirements
  - FSW Requirements should be:
    - Concise
    - Unambiguous
    - Testable
    - A shall statement
  - Help you, the developer, to know what you are implementing
  - A contract to the project on what you are implementing
    - Avoid nasty surprises when your software is finally delivered.
  - Examples:
    - The FSW shall produce spacecraft health information via telemetry.
      - Context information can also be provided as an auxiliary
        - Spacecraft health information consists of the following …
    - The FSW shall allow ground operators to change parameter values defined in the parameter dictionary.
    - The FSW shall receive files from the ground, validate them, and store them in flash memory.

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

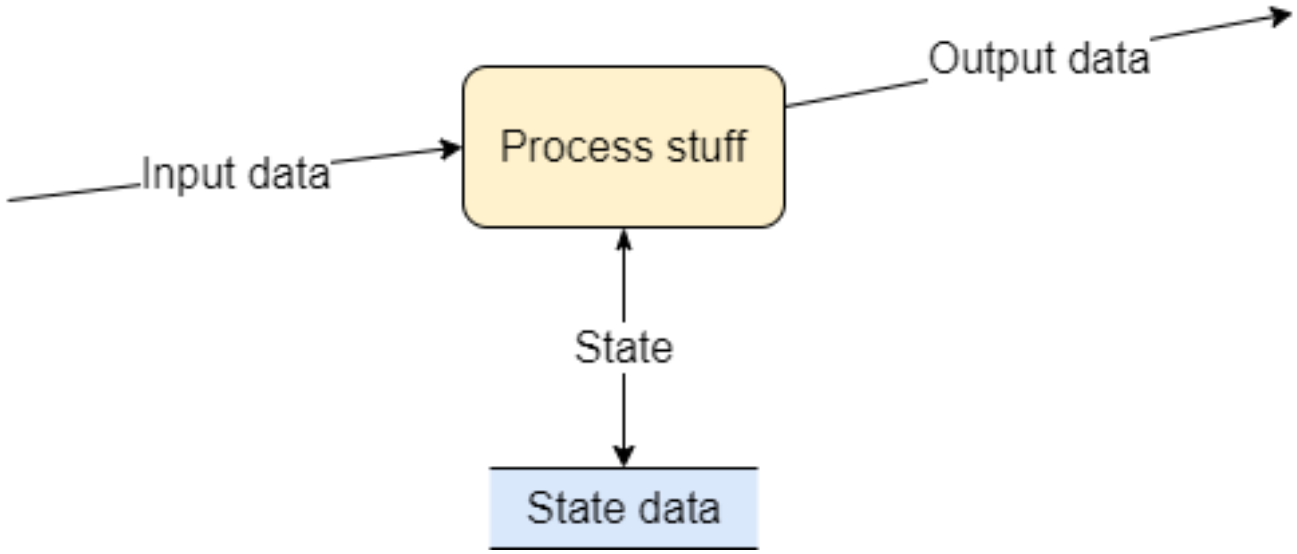
Software Requirements

- Before design and code:
  - First answer the question of What are we building?, NOT How do we implement it?
- Thoroughly understand what we building
  - Follow a process that will develop the FSW Requirements
- Structured Analysis is a process that generates a Data Flow Diagram:
  - Answers the question: What are we building
  - Naturally produces the FSW Requirements
  - Not meant to show design
  - Does not capture timing, sequence and synchronization of processes
  - Breaks the system down into smaller manageable chunks
  - A graphical diagram that is relatively easy to understand for software/system engineers
  - A clear and detailed information about the system – processes and boundaries
  - Shows the logic of the data flow

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

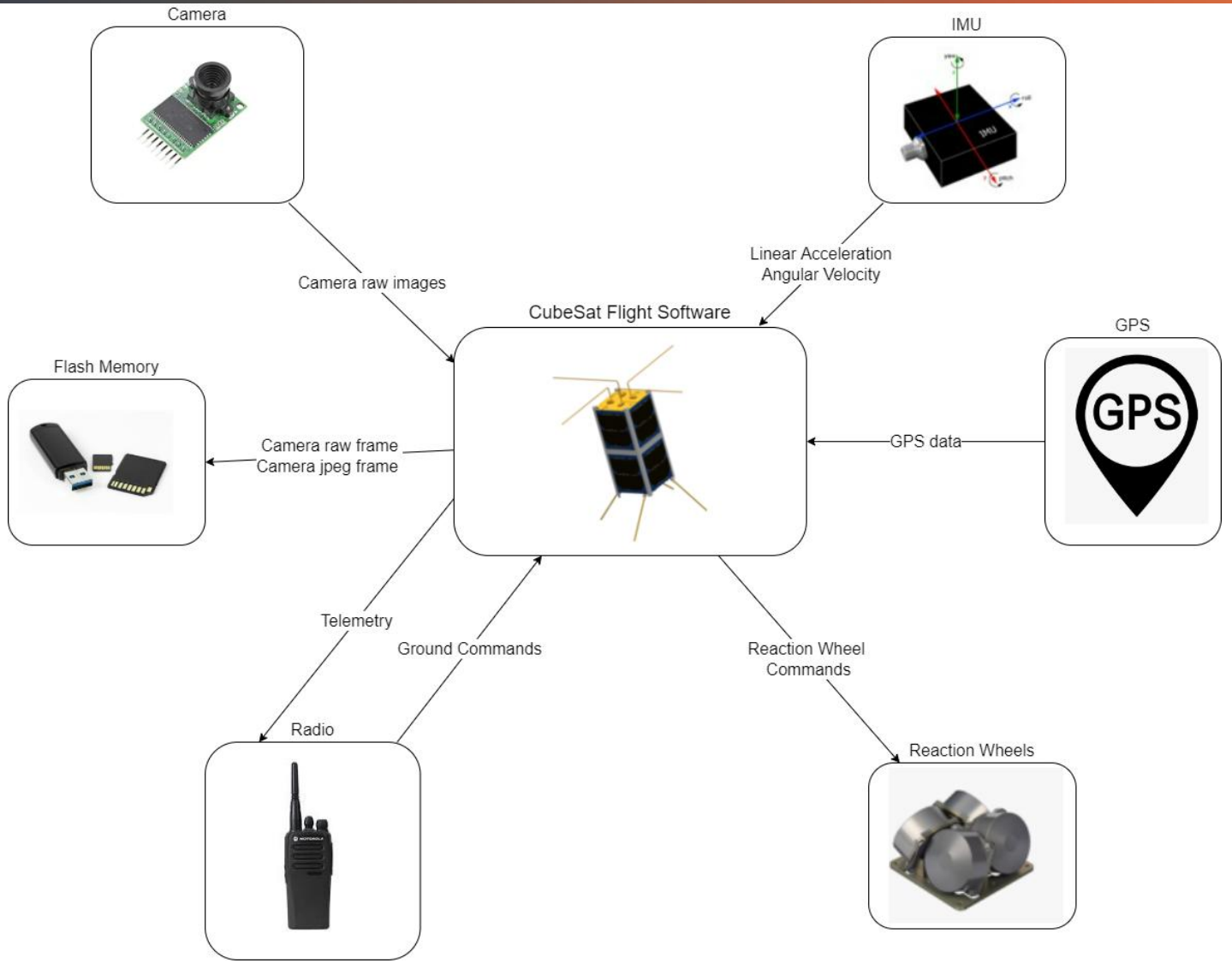
Data Flow Diagram Model

- Process (data transformation)
- Data flows
- Data stores
- Levels
  - Context diagram
    - Focus on system interfaces
  - Process decomposition
    - Overviews of system processing
  - Deeper dives
    - Detailed views



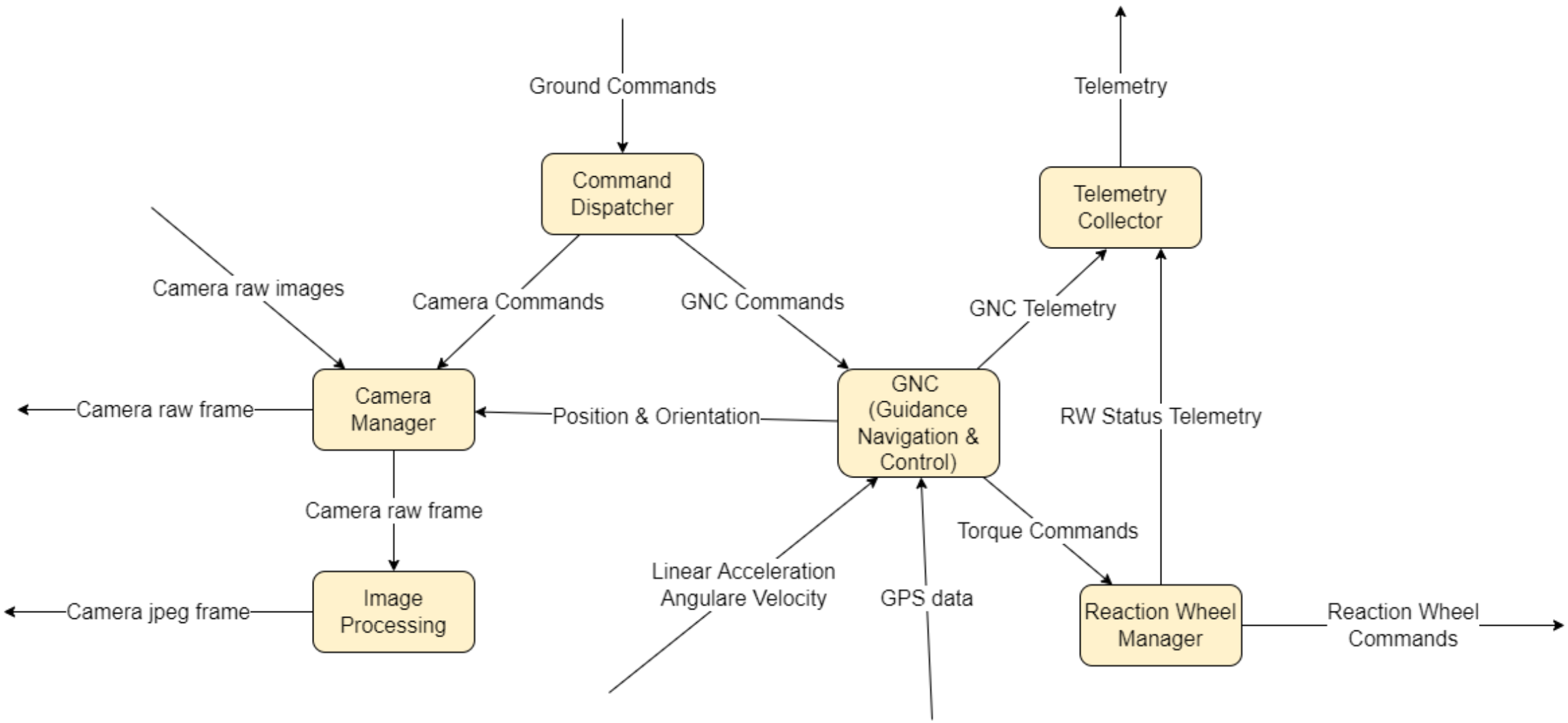
Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Flight Software Context Diagram



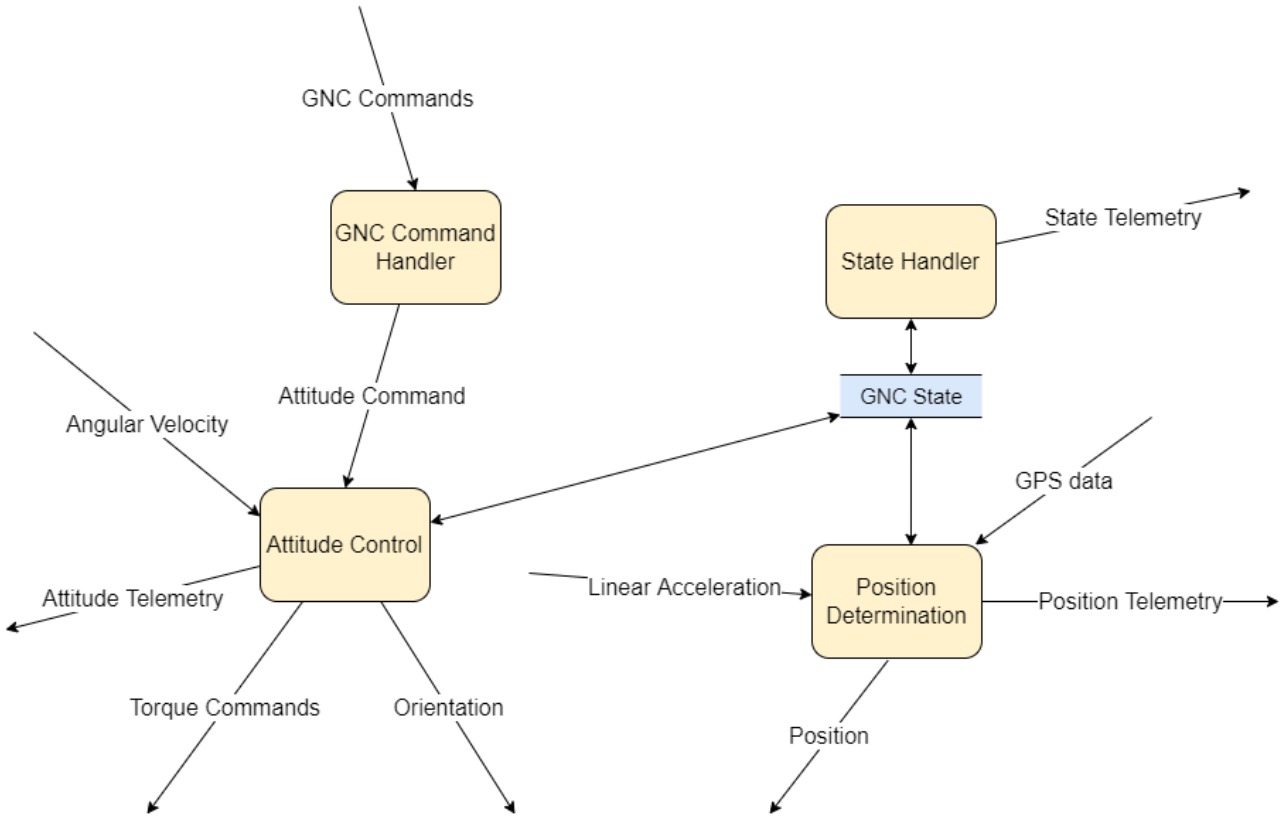
Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Top Data Flow Diagram



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

GNC Data Flow Diagram



- The FSW shall convert the GNC Command to a 3 axis pointing attitude command with an earth reference frame.
- The FSW shall send the GNC state as a telemetry channel.
- The FSW shall compute a quaternion position (a,b,c,d) from the Linear Acceleration (x,y,z) and current GPS data and send the position out as a telemetry channel.
- The FSW shall compute 3 axis torque commands (x,y,z) from a pointing attitude command and the angular velocity data.

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

From Requirements to Design

- Data Flow Diagrams (DFD's) specify:
  - Flight Software Interfaces
  - The flow of data to processes
  - The processing and transformation of the data
  - Data stores (state)
  - Used to derive flight software requirements
  - Natural transition to implementation
- Components specify:
  - The concrete implementation of the DFD
  - The periodic scheduling
  - The threads of execution (component type)
  - The component data interfaces
  - The component port types
    - Synchronous or Asynchronous

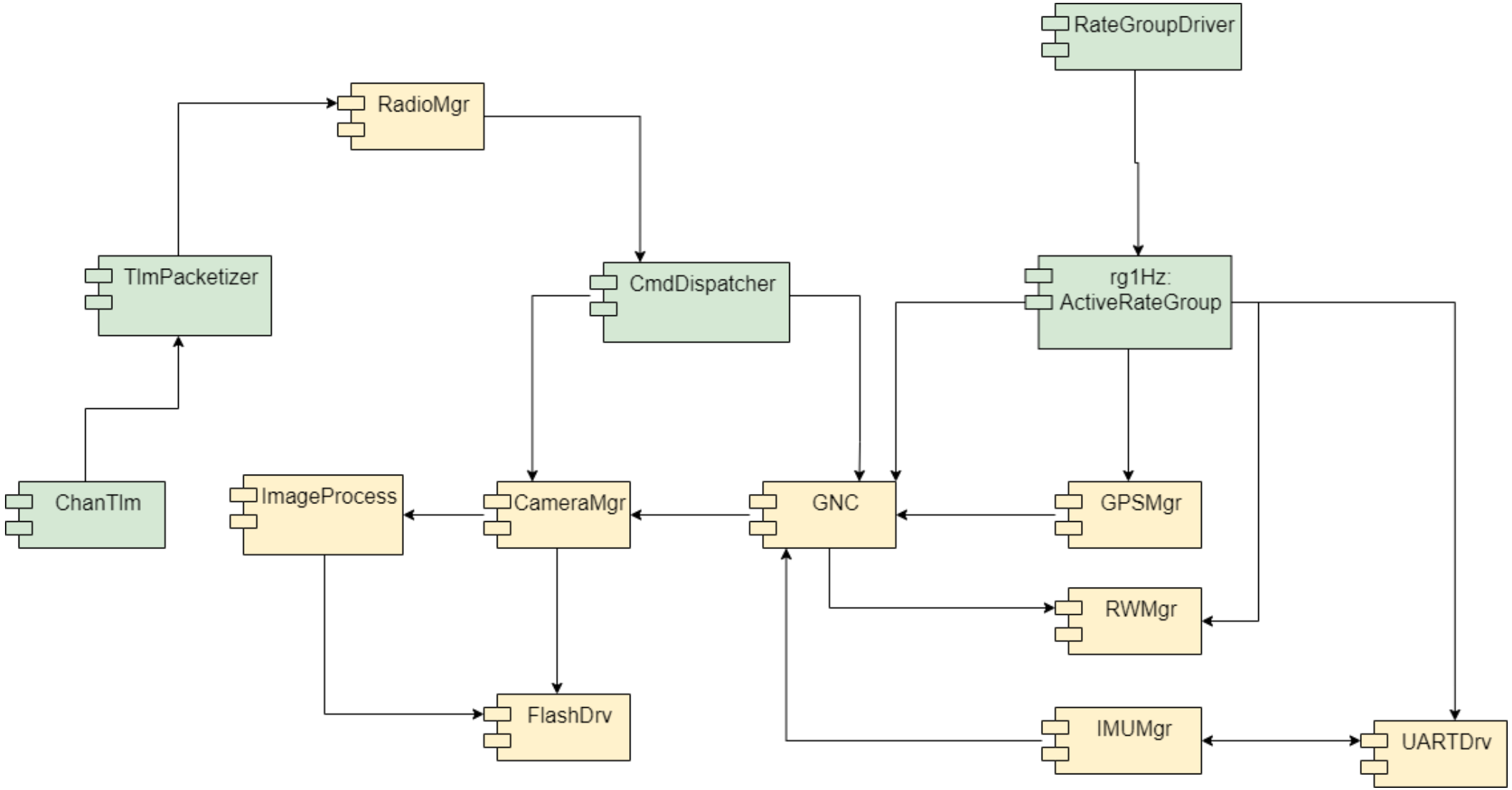
Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Design Decisions

- GPSMgr component: Implements interfacing the GPS hardware and the generation of GPS data
- RadioMgr component: Implements interfacing the Radio hardware and the processing of ground commands and flight telemetry
- IMUMgr component: Implements managing the IMU hardware and the generation of linear and angular velocity data
- UARTDrv component: Implements the low level communication to the IMU across the UART bus.
- FlashDrv component: Implements the writing of camera data products to flash
- CameraMgr component: Implements managing the Camera hardware and the Camera Manager processing
- ImageProcess component: Implements Image Processing
- GNC component: Implements the GNC processing
- F’ components ChanTlm and TlmPacketizer: Implement the collection and packetizing of telemetry data from all components
- F’ component CmdDispatcher: Implement the dispatching of ground commands to all components
- F’ components RateGroupDrv and ActiveRateGroup: Implement the periodic scheduling of components processing.

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Component Topology diagram



Jet Propulsion Laboratory
California Institute of Technology
Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Software Component State

- For each component create a crisp notion of state with a state-machine that has the following identified:
  - Discrete states
  - Signals that cause state transitions
  - Actions that get invoked on a Transition
  - State entry and exit behavior
- Encapsulate the state-machine logic within a single function or class
- Avoid a fuzzy notion of state with a collection of Boolean flags and scattered state logic.

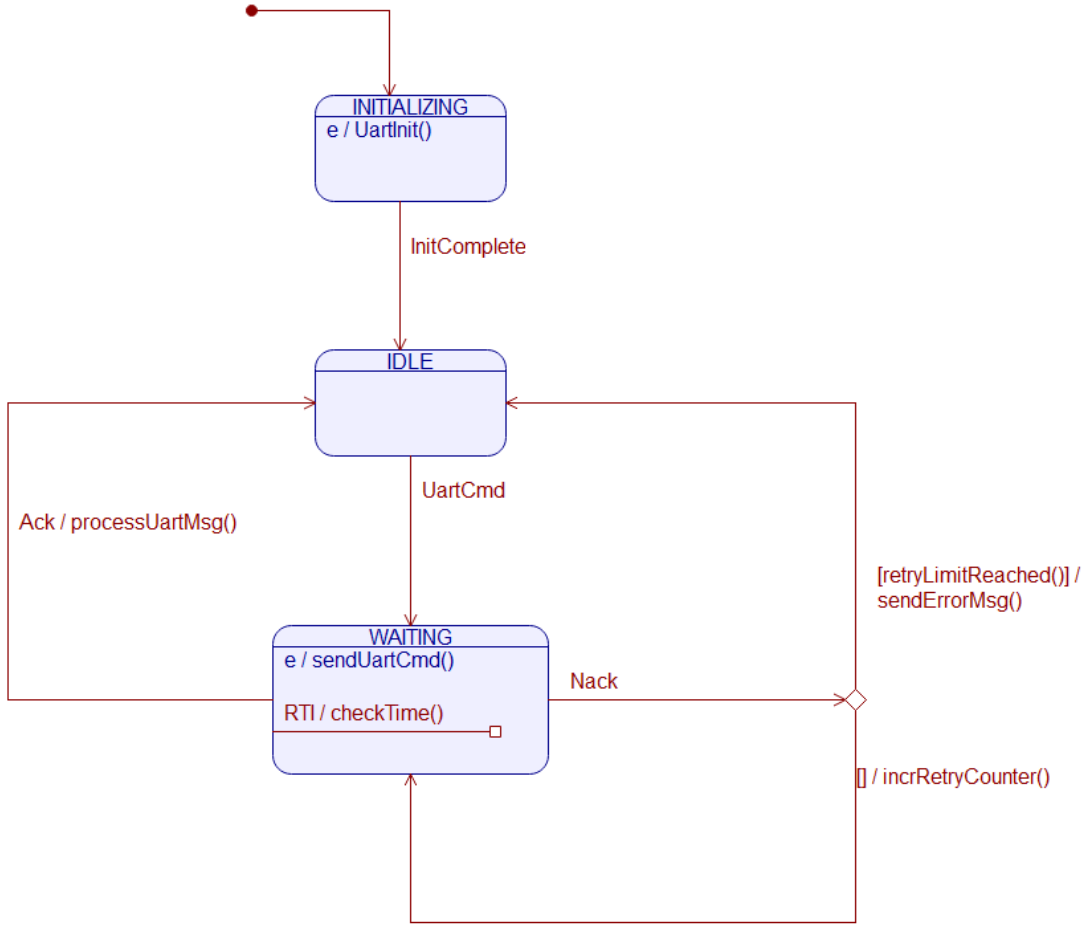
Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

UART Driver Component State-machine example

Uart Driver Requirements

- The UART Driver shall handle commands from different clients.
- The UART Driver shall process one command at a time waiting for an Ack or Nack.
- The UART Driver shall retry the command upon receiving a Nack up to a specified limit
- UART Driver shall time-out after a specified duration.

Uart Driver state machine implementation



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Embedded state machines in a component

FPP Modeling Language

State machine specification

State machine instance

```
module Drivers {

    include "UartStateMachine.fppi"

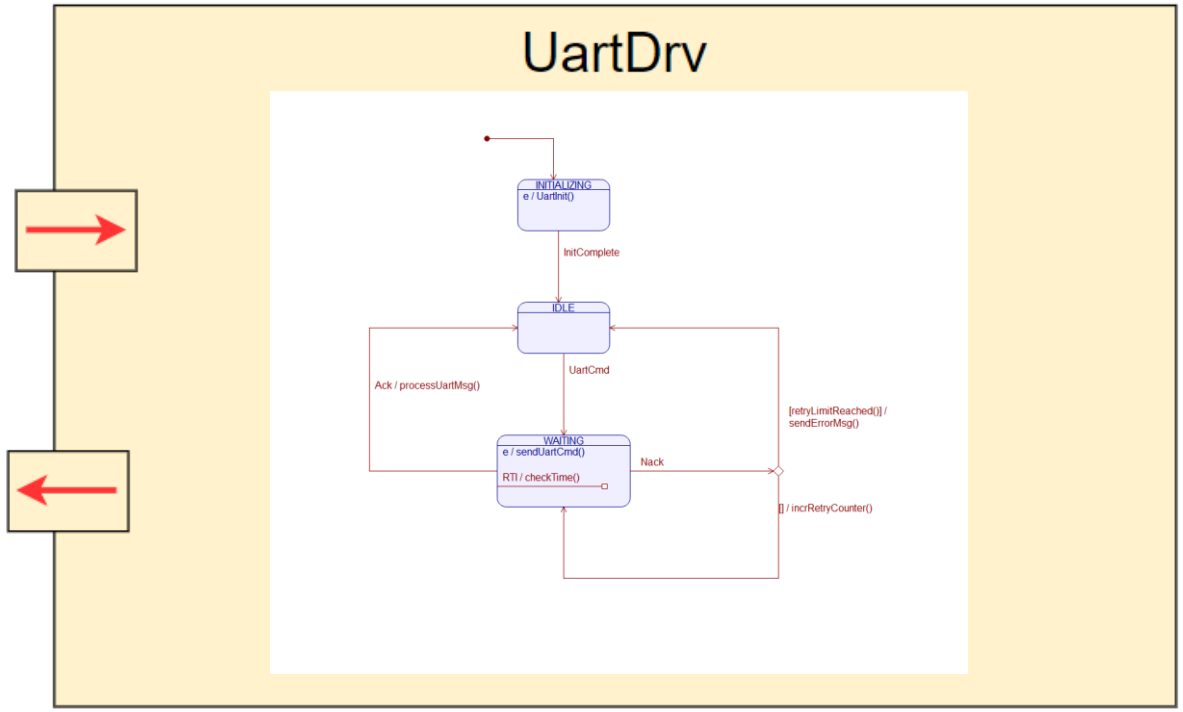
    port Rs422MsgType

    active component UartDrv {

        state machine instance uart1: UartStateMachine

        async input port dataIn: Rs422MsgType
        output port dataOut: Rs422MsgType

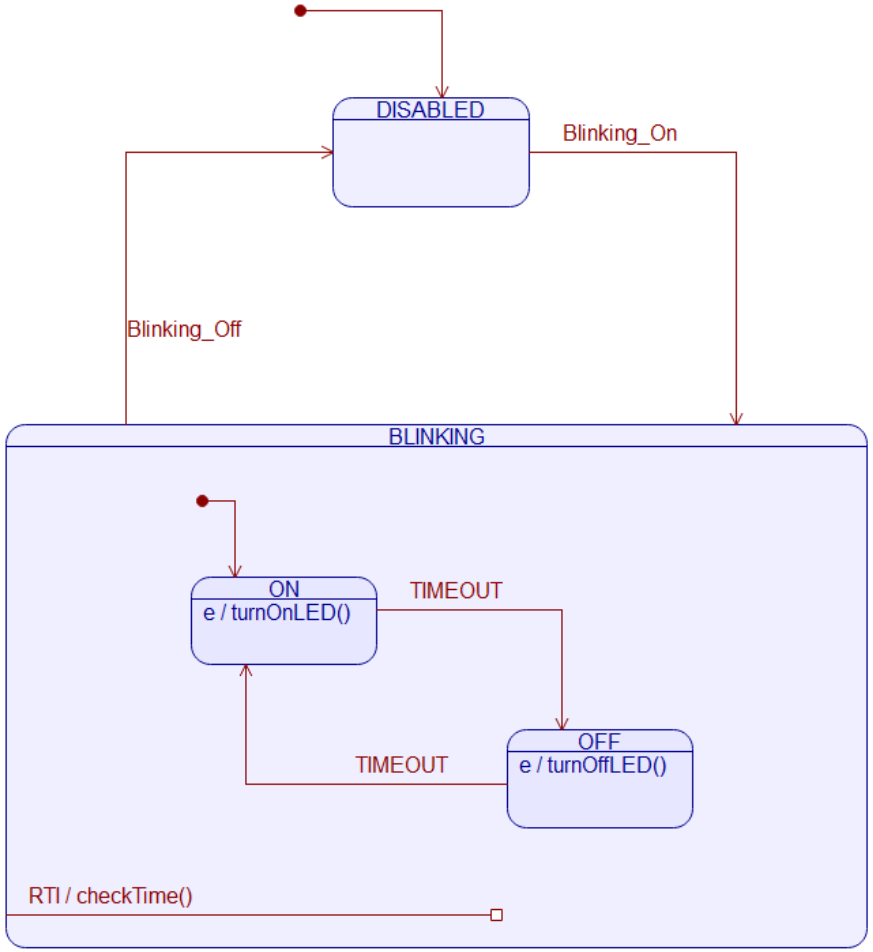
    }
}
```



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

LedBlinker Tutorial

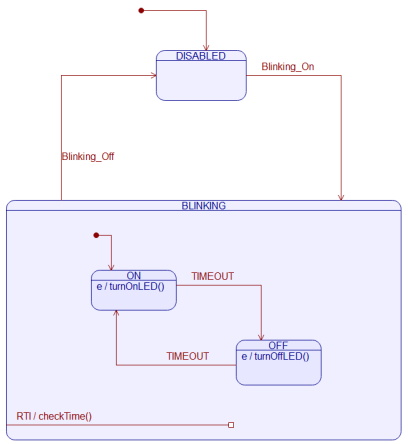
- The LedBlinker is a tutorial project to develop embedded code to control an LED
- Users will manually develop code to control an LED
- The LedBlinker tutorial can be implemented as a formal state machine using the latest F prime state machine modeling capabilities
- In the Led component Start and Stop command handlers:
  - Send signals *Blinking_On* and *Blinking_Off* to the state machine
- In the Led component Run input port handler:
  - Send signal *RTI* to the state machine
- Developer implements:
  - turnOnLED()
    Turn on the LED
  - turnOffLED()
    Turn off the LED
  - checkTime()
    Incr counter
    If counter > Threshold
    Send TIMEOUT signal
    Initialize counter



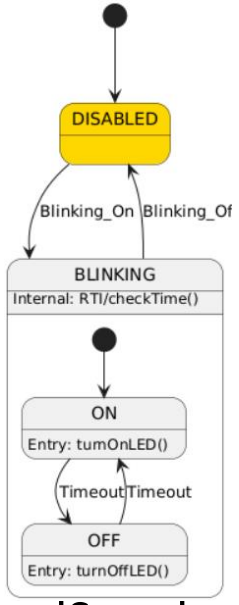
Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

LedBlinker state machine FPP generation

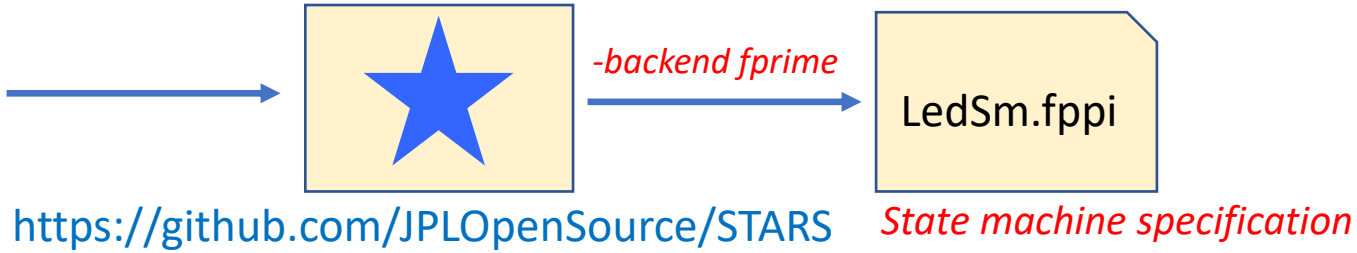
- Specify the state-machine model:
  - QM State machine tool
  - NoMagic Cameo SysML
  - PlantUML
- Use STARS to generate the FPP state machine specification



LedSm.qm



LedSm.plantuml



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Take-home message

- Understand and maintain the software architecture throughout development
- Understand what you are building before making design choices
- Generate requirements from a well understood analysis model
  - i.e. Data Flow Diagram or other models
- Capture the design as a topology model
  - Component instances
  - Component types
    - Active (Thread of execution)
    - Passive
    - Queued
  - Component interfaces
    - Data structure
    - Syncronous/Asynchronous
- Maintain a crisp notion of state for each component
  - Use a modeling tool to specify the state-machine and STARS to auto-generate the FPP OR
  - Specify the state machine directly in FPP

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.